

Análise e Modelagem de Sistemas

Resumo da I Unidade

Departamento de Ciências Exatas e Tecnológicas · UESB

1. MODELO DE SOFTWARE

1.1 Modelo de Software

Um modelo pode ser visto como uma representação idealizada de um sistema que se planeja construir. Há degradação de complexidade, necessitando de um planejamento inicial. A princípio, pode-se ver a construção de modelos como uma atividade que atrasa o desenvolvimento do software (o fator complexidade é condicionante), mas essa atividade propicia:

- O gerenciamento da complexidade inerente ao desenvolvimento de software
- A comunicação entre as pessoas envolvidas
- A redução dos custos no desenvolvimento
- A predição do comportamento futuro do sistema

1.2 Diagrama

Diagrama é uma apresentação de uma coleção de elementos gráficos que possuem significado predefinido, seguindo algum padrão lógico e construído de acordo com regras de notação bem definidas. Diagramas permitem a construção de uma representação concisa de um sistema a ser construído.

1.3 Documentação

Documentação de um modelo é formada pelo diagrama e informações textuais associadas a ele, considerando uma das perspectivas do sistema.

1.4 Modelagem de Software

A modelagem de sistemas de software consiste na utilização de notações gráficas e textuais com o objetivo de construir modelos que representam as partes essenciais de um sistema, considerando-se diversas perspectivas diferentes e complementares. A primeira abordagem usada para modelagem de sistemas de software foi o paradigma estruturado. Hoje em dia, utiliza-se o paradigma da orientação a objetos.

1.5 Paradigma Estruturado

Uso da técnica de decomposição funcional: *"Divida sucessivamente um problema complexo em subproblemas"*.

1.6 Paradigma da Orientação a Objetos

O paradigma de orientação a objetos visualiza um sistema de software como uma coleção de agentes interconectados chamados objetos. Cada objeto é responsável por realizar tarefas específicas, e é através da interação entre objetos que uma tarefa computacional é realizada. Um sistema de software orientado a objetos consiste de objetos em colaboração com o objetivo de realizar as funcionalidades deste sistema, possibilitando que a computação do sistema seja desenvolvida.

2. ENTENDENDO OS REQUISITOS

2.1 Engenharia de Requisitos

A engenharia de requisitos é o conjunto de tarefas e técnicas para entender e definir os requisitos. Tem como objetivo identificar, documentar e gerenciar as necessidades dos usuários.

- É essencial no desenvolvimento de software
- Garante que o software atenda às necessidades dos usuários
- Define um escopo claro para o projeto
- Fase inicial antes do design e implementação
- Requisitos mal definidos podem levar ao fracasso do projeto
- Evita retrabalho e desperdício de recursos
- Reduz ambiguidades e melhora a qualidade
- Melhora a comunicação entre equipe e stakeholders

2.2 Base de Trabalho

O estabelecimento de uma base de trabalho tem como objetivos criar uma base sólida para o entendimento dos requisitos e garantir que o projeto avance em direção a uma solução bem-sucedida. Alguns desafios comuns: stakeholders em locais diferentes, dificuldade em obter informações claras e consistentes e opiniões conflitantes sobre o sistema a ser executado.

2.2.1 Identificação de Stakeholders

Stakeholder é qualquer pessoa que beneficie direta ou indiretamente do sistema (exemplos: clientes, usuários, gerentes, desenvolvedores, equipe de suporte, engenheiros). Para identificar stakeholders, cria-se uma lista inicial que será expandida com base em sugestões dos próprios stakeholders.

2.2.2 Reconhecimento dos Pontos de Vista

Diferentes perspectivas: marketing foca nas funcionalidades que atraem o mercado; usuários focam na facilidade de uso e funcionalidades operacionais; infraestrutura técnica e funcionalidades invisíveis. Desafios: requisitos conflitantes entre diferentes stakeholders; necessidade de classificar e priorizar requisitos.

2.2.3 Questões Iniciais

Perguntas livres de contexto: ajudam a identificar os envolvidos e interessados no software, o benefício de uma implementação bem-sucedida e possíveis alternativas. Ex.: *Quem está por trás da solicitação deste trabalho? Quem vai usar a solução? Qual será o benefício econômico de uma solução bem-sucedida?*

Perguntas para entender melhor o problema e as percepções do cliente: *Como você caracterizaria uma "boa" saída do sistema? Qual(is) problema(s) esta solução vai resolver? Você poderia me indicar o ambiente em que a solução será usada?*

Metaperguntas: *Você é a pessoa correta para responder a estas perguntas? Suas respostas são "oficiais"? Minhas perguntas são relevantes para o problema que você tem? Estou fazendo perguntas demais? Alguma outra pessoa poderia me prestar informações adicionais? Eu deveria perguntar algo mais?*

2.2.4 Trabalho em Busca da Colaboração

Envolver todos os stakeholders no processo de definição de requisitos. Identificar áreas de consenso e conflito.

2.2.5 Rastreabilidade

Mapeamento documentado entre requisitos e outros artefatos do projeto (ex.: relacionar requisitos a casos de teste ou componentes do sistema). Benefícios: garantir que todos os requisitos sejam considerados; facilitar o controle de mudanças e a evolução do projeto.

2.3 Etapas da Engenharia de Requisitos

As etapas podem ocorrer em paralelo e devem ser adaptadas às necessidades do projeto:

- Concepção (definição inicial do problema)
- Levantamento (coleta de informações)
- Elaboração (refinamento dos requisitos)
- Negociação (resolução de conflitos)
- Especificação (documentação formal)
- Validação (verificação da consistência)
- Gestão de Requisitos (monitoramento de mudanças)

2.3.1 Concepção

Define a abrangência e a natureza do problema a ser resolvido. Define o problema e objetivos. Identifica stakeholders e comunicação inicial. Cria uma visão geral do sistema.

2.3.2 Levantamento (Elicitação de Requisitos)

Processo de coleta de informações com stakeholders sobre as necessidades e expectativas. Registro inicial dos requisitos. Combina elementos de solução de problemas, elaboração, negociação e especificação. Identificação das atividades do negócio relevantes ao sistema a ser construído. Objetivos: entender o problema e propor elementos da solução; especificar um conjunto preliminar de requisitos; servir de base para elaboração, verificação/validação, gerenciamento de conflitos, negociação, explicação e evolução dos requisitos.

Tabela 1 — Técnicas de Levantamento de Requisitos

Técnica	Descrição Resumida	Vantagens	Desvantagens	Outras Características
Entrevistas	Conversas estruturadas ou semiestruturadas com stakeholders	• Permite aprofundar temas • Flexível para explorar novas ideias • Cria rapport com o usuário	• Consome muito tempo • Pode sofrer viés do entrevistador • Difícil consolidar muitas opiniões	Formato: individual ou em grupo Melhor para: requisitos complexos e subjetivos Produto típico: atas, gravações, listas de necessidade
Questionários	Coleta de informações de múltiplos usuários simultaneamente	• Alcance grande • Baixo custo por respondente • Fácil de tabular	• Baixa profundidade • Perguntas mal interpretadas não têm correção • Baixa taxa de retorno	Formato: impresso, online Melhor para: levantamento quantitativo e opiniões amplas Cuidado: evite perguntas abertas demais
Workshops	Sessões colaborativas para definir necessidades e expectativas	• Gera consenso rápido entre grupos • Estimula criatividade e brainstorming • Reduz retrabalho por conflitos	• Difícil de organizar e conciliar agendas • Alguns participantes podem dominar a sessão • Custo alto (tempo, local, facilitação)	Formato: presencial ou remoto Comum com: JAD Produto: listas priorizadas, diagramas iniciais
Observação Direta	Acompanhamento do usuário no ambiente real de uso	• Captura o que realmente é feito • Identifica problemas implícitos e exceções • Útil para sistemas críticos ou legados	• Usuário pode mudar comportamento • Invasiva e cansativa • Não captura raciocínio interno ou regras mentais	Tipos: participante (ajuda) ou silenciosa (só observa) Melhor para: processos operacionais e tarefas repetitivas Produto: fluxograma, checklist de atividades
Prototipação	Criação de versões preliminares para coletar feedback inicial	• Valida requisitos rapidamente • Reduz ambiguidades e retrabalho futuro • Envolve o usuário de forma ativa	• Usuário pode achar que é o sistema final • Pode gerar escopo crescente • Exige ferramentas e habilidades de prototipagem	Tipos: baixa fidelidade (papel, wireframe) Produto: protótipo navegável, maquete, mockup

Técnica	Descrição Resumida	Vantagens	Desvantagens	Outras Características
Análise de Documentação	Revisão de materiais existentes para entender processos e regras de negócio	• Fonte rica e estável de informação • Baixo custo se documentação já existe • Independente da disponibilidade do usuário	• Documentos podem estar desatualizados • Pode ser tedioso e demorado • Não captura conhecimento tácito não registrado	Exemplos: manuais, formulários, regulamentos, atas de reuniões, sistemas legados Produto: glossário, regras de negócio, mapeamento de processos

Desafios do Levantamento

- Os stakeholders frequentemente não sabem o que querem do sistema a não ser em termos gerais. Podem achar difícil articular o que desejam que o sistema faça ou fazem pedidos não realistas, pois ignoram o custo de seus requisitos.
- Os stakeholders expressam os requisitos naturalmente em seus próprios termos e com o conhecimento implícito de seu trabalho. Os engenheiros de requisitos, sem experiência no domínio do cliente, devem entender esses requisitos.
- Diferentes stakeholders possuem diferentes requisitos, expressos de diferentes formas. Os engenheiros de requisitos precisam considerar todas as fontes potenciais de requisitos e descobrir pontos em comum e conflitos.
- Fatores políticos podem influenciar os requisitos do sistema (ex.: gerentes podem solicitar requisitos específicos do sistema que aumentarão sua influência na organização).
- O ambiente econômico e de negócios sobre o qual a análise é realizada é dinâmico. Ele muda inevitavelmente durante o processo de análise. Portanto, a importância de um determinado requisito pode mudar e novos podem surgir.

Boas Práticas

- Documentar todas as informações coletadas
- Validar os requisitos com os stakeholders para evitar mal-entendidos
- Usar diferentes técnicas de levantamento para obter uma visão mais ampla
- Priorizar requisitos críticos para o sucesso do projeto
- Promover colaboração ativa entre usuários e desenvolvedores

2.3.3 Elaboração

Refinamento dos requisitos levantados. Identificação de classes de análise. Definição de funcionalidades e comportamento.

2.3.4 Negociação

Resolução de conflitos entre requisitos. Priorização e viabilidade dos requisitos. Busca de consenso de stakeholders.

2.3.5 Especificação

Documentação formal dos requisitos. Pode incluir diagramas e descrições textuais. Serve como referência para o desenvolvimento. É o processo de escrever os requisitos de usuário e de sistema em um documento de sistemas. Os requisitos devem ser claros, inequívocos, fáceis de entender, completos e consistentes.

Documentos de requisitos: declaração oficial do que deve ser implementado. É essencial quando: o sistema tem o seu desenvolvimento terceirizado; times diferentes desenvolvem partes diferentes do sistema; ou uma análise detalhada dos requisitos é obrigatória.

Usuários do documento de requisitos:

- Clientes do sistema: especificam os requisitos e os leem para conferir se satisfazem suas necessidades; especificam mudanças nos requisitos

- Gerentes: usam o documento para planejar uma proposta para o sistema e planejar o seu processo de desenvolvimento
- Engenheiros de sistema: usam os requisitos para compreender qual sistema deve ser desenvolvido
- Engenheiros de teste do sistema: usam os requisitos para desenvolver testes de validação do sistema
- Engenheiros de manutenção do sistema: usam os requisitos para entender o sistema e as relações entre suas partes

2.3.6 Validação

Verifica se os requisitos estão corretos. É o processo de conferir se os requisitos definem o sistema que o cliente realmente quer. Erros em um documento de requisitos podem levar a grandes custos quando esses problemas são descobertos durante o desenvolvimento ou após o sistema entrar em produção.

Conferências necessárias:

- Validade (atende necessidades dos usuários?)
- Consistência (sem conflitos)
- Completude (todas funções/restrições)
- Realismo (viabilidade técnica/orçamentária)
- Verificabilidade (testes possíveis)

Métodos: revisões, prototipagem, casos de teste.

2.3.7 Gestão de Requisitos

Os requisitos dos sistemas de software mudam. Controle de mudanças nos requisitos. Monitoramento do impacto das alterações. Uso de ferramentas como Jira, Trello e Git.

Requisitos duradouros: são aqueles associados às atividades centrais da organização (mudam lentamente).

Requisitos voláteis: são propensos à mudança. Normalmente, estão associados às atividades de apoio que refletem como a organização faz o seu trabalho.

Como os requisitos evoluem, é preciso acompanhar cada um e manter vínculos entre os requisitos dependentes para que se possa avaliar o impacto das mudanças. Envolve o processamento de grande quantidade de informações sobre os requisitos.

3. UNIFIED MODELING LANGUAGE (UML)

3.1 Modelagem de Requisitos

A modelagem de requisitos usa uma combinação de notações gráficas e textuais com o objetivo de construir modelos que representam as partes essenciais de um sistema, considerando-se as perspectivas diversas e complementares.

3.2 UML

Os diagramas da UML se tornaram o padrão da engenharia de software para modelar graficamente as relações e os comportamentos entre elementos do modelo de análise.

A UML é a **linguagem padrão para visualizar, especificar, construir e documentar** os artefatos de software de um sistema. Unificação de diversas notações anteriores.

UML é: uma linguagem visual; independente de linguagem de programação; independente de processo de desenvolvimento.

UML não é: uma linguagem de programação; uma técnica de modelagem.

3.3 Diagramas da UML 2.0

Um diagrama na UML é uma apresentação de uma coleção de elementos gráficos que possuem um significado predefinido. No contexto de desenvolvimento de software, correspondem a desenhos gráficos que seguem algum padrão lógico.

Diagramas Estruturais: Diagrama de Objetos, Diagrama de Classes, Diagrama de Pacotes, Diagrama de Estrutura Composta, Diagrama de Componentes, Diagrama de Implantação.

Diagramas Comportamentais: Diagrama de Casos de Uso, Diagrama de Atividades, Diagrama de Transições de Estados, Diagrama de Sequência, Diagrama de Colaboração, Diagrama de Temporização, Diagrama de Visão Geral da Interação.

3.4 Artefatos de Software

Artefatos de software são diversos documentos criados no processo de desenvolvimento que utilize a UML como linguagem de modelagem, podendo ser textuais ou gráficos. Os artefatos gráficos produzidos no desenvolvimento de um sistema OO são definidos através dos diagramas da UML.

4. MODELAGEM BASEADA EM CENÁRIOS

É uma técnica que representa todos os possíveis usos de um sistema, conforme percebidos por um observador externo (o usuário). O principal artefato dessa modelagem é o **Modelo de Casos de Uso (MCU)**.

4.1 Modelo de Casos de Uso (MCU)

É uma representação das funcionalidades externamente observáveis do sistema e dos elementos externos ao sistema que interagem com ele. Representa os requisitos funcionais do sistema. Também direciona diversas das atividades posteriores do ciclo de vida do sistema de software e força os desenvolvedores a moldar o sistema de acordo com as necessidades do usuário. Tem como objetivo **capturar e comunicar os requisitos funcionais do sistema** (o que o sistema faz) de forma simples e compreensível para todos os envolvidos (técnicos e não-técnicos).

Utilidade dos casos de uso:

- Equipe de clientes: realizam validação, aprovando e entendendo o que o sistema deverá fazer
- Equipe de desenvolvedores: ponto de partida para refinar requisitos de software; podem seguir um desenvolvimento dirigido a casos de uso; o designer (projetista) usa para encontrar classes, enquanto os testadores usam como base para casos de teste

4.2 Composição do MCU

O modelo de casos de uso de um sistema é composto por duas partes: gráfica e textual. O **Diagrama de Casos de Uso** é o diagrama da UML utilizado na modelagem gráfica. Permite dar uma visão global e de alto nível do sistema. Modelagem dinâmica de sistema em alto nível de abstração. Relaciona o conjunto de situações de processamento do software (casos de uso) e os elementos externos que interagem com o software. É também chamado de diagrama de contexto.

4.3 Elementos Principais do MCU

4.3.1 Casos de Uso

Uma funcionalidade do software: atômica, completa (não uma fração), externamente perceptível, sem qualquer referência a como ela é executada. Sequência de interações entre o sistema e o ator (troca de informações entre eles). Representa o que um ator quer que o sistema faça. Especificação de uma sequência de interações entre um sistema e os agentes externos. Define parte da funcionalidade, sem revelar a estrutura e o

comportamento internos deste sistema. **Escopo do sistema:** delimitado de forma clara ao definir as funções dos atores e dos casos de uso.

4.3.2 Ator

Elemento externo (atores não fazem parte do sistema) que interage com o sistema. Representa um conjunto coerente de papéis que os usuários de casos desempenham quando interagem com o sistema. Corresponde a um papel representado em relação ao sistema (um mesmo indivíduo pode corresponder a diferentes atores). O nome dado a um ator deve lembrar o seu papel, em vez de lembrar quem o representa. Definem o ambiente do sistema, se comunicando via envio/recepção de mensagens do sistema.

Quantos atores? Todo software interage com o meio externo, logo, há no mínimo um ator. Quando a interação com o meio externo pode ocorrer de mais de uma forma (formas distintas de interação), há mais de um ator, demandando a implementação de mais de uma interface.

- **Participação ativa:** o ator inicia a interação com o sistema, fornecendo dados ou comandos
- **Participação passiva:** ator apenas recebe informações do sistema, sem iniciar ações ou fornecer entrada

4.3.3 Associação entre Ator e Caso de Uso

Participação da entidade modelada pelo ator em um caso de uso. Um diagrama de casos de uso não deve apresentar nem ator isolado e nem caso de uso isolado. Um ator sempre está associado a pelo menos um caso de uso e um caso de uso, mesmo que indiretamente, está associado a pelo menos um ator.

Um ator associado a um caso de uso: atuação de um único elemento externo (restritas a um tipo específico de ator).

Mais de um ator associado a um caso de uso: atuação conjunta (exige atuação coordenada de dois ou mais atores) ou alternativa (acessíveis de forma isolada por qualquer um deles). A notação UML não diferencia sintaticamente conjunção de disjunção.

Extensão: estabelece uma relação em que um dos casos de uso tem seu comportamento estendido através do comportamento definido em outro caso de uso (seta aponta do caso extensão para o caso base). Denota **opcionalidade:** quando o caso principal ocorre, o caso de uso auxiliar poderá ou não ocorrer.

Inclusão: estabelece que parte do comportamento inerente a um caso de uso está definida em outro caso de uso. Permite evitar repetições na modelagem pois extrai um conjunto de passos comum a vários casos de uso em um único caso de uso "incluído". Denota **obrigatoriedade:** quando o caso de uso principal ocorre, o caso de uso auxiliar também ocorrerá.

Generalização — casos de uso: estabelece uma relação de especialização entre dois casos de uso, onde um corresponde a um comportamento genérico e o outro a uma especialização deste para alguma situação específica (seta aponta da especialização para o comportamento genérico). O caso pai pode ser abstrato. Ex: realizar pagamento — com cartão de crédito ou dinheiro.

Generalização — atores: dos dois atores envolvidos em uma generalização, um corresponde a um papel genérico e outro a uma especialização deste (seta aponta da especialização para o papel genérico).

5. DESCRIÇÃO DOS CASOS DE USO

Um modelo de casos de uso típico é formado de vários casos de uso. Cada caso de uso é definido através da descrição textual das interações que ocorrem entre os elementos externos do sistema.

5.1 Identificação dos Elementos do MCU

Atores e casos de uso são identificados a partir de informações coletadas no levantamento de requisitos. A quantidade de casos de uso e atores depende da complexidade do sistema, não havendo uma regra geral. A

identificação de atores e de casos de uso são atividades que se intercalam.

5.1.1 Identificação de Atores

Como um ator é todo elemento externo que interage com o sistema, fontes e destinos das informações a serem processadas são atores em potencial. O analista deve identificar as áreas da empresa que serão afetadas ou utilizarão o sistema, fontes de informações a serem processadas e os destinos das informações geradas pelo sistema.

Perguntas úteis para identificar atores:

- Que órgãos, empresas ou pessoas (cargos) irão utilizar o sistema?
- Que outros sistemas irão se comunicar com o sistema?
- Alguém deve ser informado de alguma ocorrência no sistema?
- Quem está interessado em um certo requisito funcional do sistema?

5.1.2 Identificação de Casos de Uso

Caso de uso primário: representa os objetivos dos atores.

Caso de uso secundário: não traz benefício direto para os atores, mas é necessário para que o sistema funcione adequadamente. São menos importantes que os primários. Exemplos: manutenção de cadastros, manutenção de usuários, gerenciamento de acesso, manutenção de informações provenientes de outros sistemas.

Perguntas úteis para identificar casos de uso:

- Quais são as necessidades e objetivos de cada ator em relação ao sistema?
- Que informações o sistema deve produzir?
- O sistema deve realizar alguma ação que ocorre regularmente no tempo?
- Para cada requisito funcional, existe um (ou mais) caso(s) de uso para atendê-lo?

Outras técnicas de identificação: caso de uso "oposto", caso de uso que precede/sucedee a outro caso de uso, caso de uso temporal, caso de uso relacionado a uma condição interna.

5.2 Construção do DCU

Quanto mais fácil for a leitura do diagrama representando casos de uso, melhor. Se o sistema não for tão complexo, pode ser criado um único DCU. Em sistemas complexos, representar todos os casos de uso em um único DCU talvez o torne ilegível. Solução: criar vários diagramas e agrupá-los em pacotes (separação por: casos de uso para um ator, por ciclo de desenvolvimento, por área da empresa). Além disso, é útil e recomendada a utilização do **retângulo de fronteira** para delimitar e separar visualmente casos de uso e atores.

5.3 Documentação

Documentação dos atores: uma breve descrição para cada ator deve ser adicionada ao MCU. O nome de um ator deve lembrar o papel desempenhado por ele.

Documentação dos casos de uso: a UML não define um padrão para descrição textual dos casos de uso de um sistema e, por conta disso, há diversos estilos de descrição possíveis (numerada, livre, tabular etc.). É necessário que a equipe padronize o seu estilo de descrição.

Boas práticas na documentação de casos de uso:

- Começar o nome do caso de uso com um verbo no infinitivo (para indicar um processo ou ação)
- Descrever os passos sempre na forma sujeito + predicado (deixar explícito quem é o agente da ação)
- Não descrever como o sistema realiza internamente um passo de caso de uso

- Dar nomes seguindo perspectiva do ator primário, focando em seu objetivo (ex: registrar pedido, alugar filme)
- Manter a descrição de cada caso de uso no nível mais simples possível

Seções encontradas na documentação dos casos de uso:

1. Identificador
2. Nome: título do caso de uso, iniciado com verbo no infinitivo
3. Descrição: resumo do que o caso de uso faz e qual objetivo ele atende
4. Importância: nível de relevância do caso de uso para o sistema (ex.: alta, média, baixa)
5. Ator Primário: aquele que inicia o caso de uso para alcançar um objetivo
6. Atores Secundários: atores que dão suporte ao caso de uso, sem iniciá-lo
7. Pré-condições: condições que devem ser verdadeiras antes do caso de uso começar
8. Sumário: visão geral resumida do caso de uso, incluindo breve descrição do objetivo e do que é feito
9. Fluxo principal (fluxo básico): descrição da sequência de passos usual; descreve o que normalmente acontece quando o caso de uso é utilizado; toda descrição de caso de uso deve ter um fluxo principal
10. Fluxos alternativos: descrevem o que acontece quando o ator opta por utilizar o caso de uso de uma forma diferente do fluxo principal para alcançar o seu objetivo; também utilizados para descrever situações com maneiras diversas alternativas
11. Pós-condições: estado do sistema após o caso de uso terminar (em sucesso ou falha)
12. Regras do Negócio: políticas, condições ou restrições que devem ser consideradas na execução dos processos; descrevem a maneira pela qual a organização funciona. Ex: o valor total de um pedido é igual à soma dos totais dos itens do pedido acrescido de 10% de taxa de entrega
13. Referências cruzadas (para requisitos não funcionais): ligações entre o caso de uso e requisitos não funcionais (ex.: performance, segurança)
14. Histórico: registro de versões, datas, autores e alterações feitas na descrição
15. Notas de Implementação: observações técnicas sobre como o caso de uso será implementado

6. MODELAGEM BASEADA EM CLASSE

A modelagem baseada em classes é uma técnica do paradigma orientado a objetos que representa um sistema de software como uma coleção de classes interconectadas. Modelar classes não é desenhar diagramas, é entender o domínio. Não existe técnica única perfeita, sendo essencial a combinação de abordagens.

Classe: cada classe descreve um conjunto de objetos com atributos (dados) e operações (comportamentos) comuns. Cada classe representa algo relevante no domínio do problema. Identificação de classes é iterativa. Apesar de todas as vantagens que a OO pode trazer ao desenvolvimento de software, um problema fundamental ainda persiste: identificar corretamente e completamente objetos (classes), atributos e operações.

6.1 Construção do Modelo de Classes

Após a identificação de classes, deve verificar a consistência entre as classes para eliminar incoerências e redundâncias. Depois, deve-se começar a definir o mapeamento das responsabilidades e colaboradores de cada classe para os elementos do diagrama de classes.

Definição de propriedades: uma responsabilidade de conhecer é mapeada para um ou mais atributos. Operações de uma classe: modo mais detalhado de explicitar as operações de fazer; contribuição da classe para uma tarefa mais complexa representada por um caso de uso.

Definição de associações: a existência de relacionamentos se dá pela colaboração de classes (um objeto precisa conhecer o outro para que possa ser feita requisições). Para criar associações, é necessário verificar os

colaboradores de uma classe. O mesmo raciocínio se aplica a: associações reflexivas (relacionamento de uma classe com ela mesma), ternárias (relacionamento simultâneo entre três classes diferentes) e agregações (relação "todo-parte" em que a parte pode existir sem o todo).

6.2 Técnicas de Identificação de Classes

6.2.1 *Análise Textual de Abbott*

Baseada na análise de descrições textuais do sistema. **Ideia central: substantivos → possíveis classes.** Fontes de informação utilizadas: documento de requisitos, modelos do negócio, glossários, conhecimento sobre o domínio. Estratégia: identificar termos da narrativa de casos de uso e documento de requisitos que podem sugerir classes, atributos, operações. Destacar substantivos e adjetivos e remover sinônimos, permanecendo os nomes mais significativos para o domínio.

Passos: (1) Ler os textos do problema; (2) Identificar substantivos; (3) Filtrar candidatos — remover termos irrelevantes, sinônimos, elementos muito genéricos (ex: "sistema"); (4) Refinar o conjunto de classes.

Vantagens: é de aplicação bastante simples; boa para início. **Desvantagens:** pode gerar excesso de classes; depende da qualidade do texto; variações linguísticas e formas de expressar uma mesma ideia são bastante numerosas em linguagem natural.

6.2.2 *Análise de Casos de Uso*

É um caso particular da análise textual, utilizando o MCU como ponto de partida. Premissa: um caso de uso corresponde a um comportamento específico do sistema; esse comportamento somente pode ser produzido por objetos que o compõem. A realização de um caso de uso é responsabilidade de um conjunto de objetos que colaboram para produzir o resultado dele.

Procedimento: o modelador estuda a descrição textual de cada caso de uso para identificar classes candidatas. São analisados para cada caso de uso: fluxos principal, alternativos e de exceção, pré-condições e pós-condições. O modelador tenta identificar classes que possam fornecer o comportamento do caso de uso analisado. Quando todos os casos de uso forem analisados, a maioria das classes terá sido identificada.

6.2.3 *Padrões de Análise*

Após produzir diversos modelos para domínios parecidos, o modelador começa a identificar características comuns entre eles. Uso de soluções já conhecidas. Reutilizar modelos existentes.

6.2.4 *Identificação Dirigida a Responsabilidades*

Baseada no conceito de **responsabilidade**: "Quem deve fazer isso?". Responsabilidade é alguma coisa que um objeto conhece ou sabe fazer (sozinho ou pedindo ajuda). Se um objeto tem uma responsabilidade com a qual não pode cumprir sozinho, ele deve requisitar colaborações de outros objetos. Uma classe deve: **fazer algo** (ação) e **saber algo** (informação).

Ênfase na identificação de classes a partir de seus **comportamentos relevantes** para o sistema. O método enfatiza o encapsulamento da estrutura e do comportamento dos objetos (identificação das responsabilidades de uma classe que são úteis externamente à mesma). Os detalhes internos (como a classe faz para cumprir suas responsabilidades) devem ser abstraídos.

Cartão CRC (Class, Responsibility, Collaboration): ferramenta de modelagem dividida em partes: parte superior — nome da classe; coluna esquerda — responsabilidades da classe; coluna direita — outras classes que colaboram com a classe para que ela cumpra suas responsabilidades.

Exemplo de Cartão CRC:

Aluno	
Responsabilidades	Colaboradores

7. MODELAGEM FUNCIONAL (VISÃO PROCEDURAL)

7.1 Modelagem Funcional

O modelo funcional lida com dois elementos de processamento de aplicação, cada um representando um nível diferente de abstração procedural.

- **Funcionalidade observável:** funções iniciadas diretamente pelo usuário — aquilo que ele vê e recebe como resultado. Exemplo: cálculo de parcela de empréstimo em um app financeiro.
- **Operações de classes:** processamento interno realizado pelas operações das classes de análise, manipulando atributos e colaborando para produzir comportamentos esperados.

7.2 Diagrama de Atividades

O diagrama de atividades da UML pode ser utilizado para representar detalhes do processamento em qualquer nível de abstração procedural. Devem ser utilizados quando a funcionalidade é relativamente complexa. Voltado a detalhar o comportamento de um programa: descrevê-lo quando em execução, em alto ou em baixo nível de abstração. Funciona como complemento ao caso de uso, representando graficamente o fluxo de interação em um cenário específico.

7.3 Elementos Principais do Diagrama de Atividades

7.3.1 Elementos em Fluxos Sequenciais

- **Ação:** unidade de modelagem de comportamento atômica (não admite particionamento); algo que é executado de forma ininterrupta.
- **Atividade:** unidade de modelagem de comportamento não atômica; pode ser composta por ações ou por outras atividades.
- **Fluxo de controle:** "seta" apontando de um procedimento para o procedimento seguinte; representa a passagem do controle da execução de uma ação/atividade para outra; estabelece as sequências possíveis de execução.
- **Estado inicial:** estabelece o início da execução de uma atividade.
- **Estado final:** estabelece o final da execução de uma atividade.
- **Ponto de ramificação (nó decisão):** permite fluxos de controle alternativos, condicionados por expressões booleanas (equivalente a if, if-else e switch/case). Possui uma única transição de entrada e várias transições de saída, cada uma associada a uma condição de guarda. Quando o fluxo chega ao ponto, uma e somente uma das condições deve ser verdadeira. Pode haver uma transição com [else]. Representado por um losango com uma entrada e "n" saídas.
- **Ponto de união (nó fusão):** funde um conjunto de fluxos em um único. Não tem semântica de sincronização; apenas reúne um conjunto de elementos sintáticos num único. Reúne diversas transições que, direta ou indiretamente, têm um ponto de ramificação em comum.

7.3.2 Elementos em Fluxos Paralelos

- **Barra de bifurcação (fork):** recebe uma transição de entrada e cria dois ou mais fluxos de controle paralelos; cada fluxo é executado independentemente e em paralelo com os demais.
- **Barra de junção (join):** recebe duas ou mais transições de entrada e une os fluxos de controle em um único fluxo. Objetivo: sincronizar fluxos paralelos. A transição de saída somente é disparada quando todas as transições de entrada tiverem sido disparadas.

7.4 Raias de Natação (Swim Lanes)

Algumas vezes, as atividades de um processo podem ser distribuídas por vários agentes que o executarão (ex.: processos de negócio de uma organização). Isso pode ser representado através de raias de natação (swim lanes).

- As raias de natação dividem o diagrama de atividade em compartimentos
- Cada compartimento contém atividades que são realizadas por uma entidade (ator, departamento, sistema)

7.5 Aplicações do Diagrama de Atividades

Modelar o processo do negócio: modelagem também é um processo de entendimento. O desenvolvedor constrói modelos para entender melhor um problema. O enfoque está em entender o comportamento do sistema no decorrer de diversos casos de uso (processos de negócio) — como determinados casos de uso do sistema se relacionam no decorrer do tempo.

Modelar a lógica de um caso de uso: a realização de um caso de uso requer que alguma computação seja realizada, que pode ser dividida em atividades. Em situações com condições ou bifurcações (ex: "Passo P ocorre até que a condição C seja verdadeira"), é interessante complementar a descrição do caso de uso com um diagrama de atividade. Os fluxos principal, alternativos e de exceção podem ser representados em um único diagrama de atividade. O diagrama complementa e não substitui a descrição textual. Casos de uso são descritos na perspectiva dos atores, enquanto diagramas de atividade descrevem atividades internas ao sistema.

Modelar a lógica de uma operação: quando um sistema é adequadamente decomposto em seus objetos, a maioria das operações são bastante simples e não necessitam de modelagem gráfica. No entanto, pode haver a necessidade de descrever a lógica de uma operação mais complexa (ex.: implementação de regras de negócio).

8. TABELA COMPARATIVA: CASOS DE USO × CLASSES × ATIVIDADES

Critério	Diagrama de Casos de Uso	Diagrama de Classes	Diagrama de Atividades
O que representa	Funcionalidades do sistema e interações com elementos externos (atores)	Estrutura estática do sistema: classes, atributos, operações e relacionamentos	Fluxo de controle e sequência de ações/atividades (comportamento dinâmico)
Perspectiva	Dinâmica, mas em alto nível de abstração (o que o sistema faz, não como)	Estática (estrutura do domínio)	Dinâmica (como o sistema se comporta em execução)
Nível de abstração	Alto (visão de contexto / "caixa-preta")	Médio a baixo (detalha estrutura interna)	Variável (alto para processos de negócio; baixo para lógica de operações)
Principal artefato	Modelo de Casos de Uso (MCU) — diagrama + descrições textuais	Modelo de Classes — diagrama com classes, atributos, operações, associações	Diagrama de Atividades — fluxograma comportamental
Elementos principais	Ator, caso de uso, associações (include, extend, generalização), retângulo de fronteira	Classe, atributo, operação, associação, agregação, composição, generalização, associações reflexivas/ternárias	Ação, atividade, fluxo de controle, estado inicial/final, ponto de ramificação/união, fork/join, raias de natação
Quando usar	• Levantamento de requisitos funcionais • Comunicação com stakeholders não técnicos • Definir escopo do sistema • Validar o que o sistema deve fazer	• Análise e projeto da estrutura do domínio • Após identificação de classes via casos de uso ou análise textual • Definir atributos, operações e relacionamentos	• Funcionalidade complexa que exige detalhamento • Complementar descrição de caso de uso • Modelar processos de negócio (vários casos de uso) • Descrever lógica de operações complexas
Vantagens	• Fácil entendimento por não técnicos • Comunicação eficaz com clientes • Delimita escopo claramente • Base para testes e identificação de classes	• Representa estrutura reutilizável • Linguagem padrão para OO • Permite identificar atributos e operações • Base para implementação	• Representa fluxos alternativos e paralelos claramente • Único diagrama para fluxos principal + alternativos + exceção • Swim lanes mostram responsabilidade por entidade • Complementa texto do caso de uso
Desvantagens	• Não detalha comportamento interno • Pode se tornar ilegível se muitos casos de uso • Diferentes estilos de descrição textual (UML não padroniza) • Não diferencia sintaticamente conjunção de disjunção	• Complexo para leigos • Identificação de classes é desafiadora (técnicas são necessárias) • Pode gerar excesso de classes (ex: análise textual) • Depende da qualidade dos textos fonte	• Pode ficar complexo se muitos fluxos paralelos • Não substitui descrição textual do caso de uso • Pode ser desnecessário para operações simples • Exige conhecimento de notações (fork, join, guardas)

Critério	Diagrama de Casos de Uso	Diagrama de Classes	Diagrama de Atividades
Relação com outras visões	• Alimenta a identificação de classes (via análise de casos de uso) • Diagrama de atividades complementa casos de uso complexos	• Classes realizam comportamentos definidos nos casos de uso • Operações contribuem para tarefas representadas por casos de uso	• Complementa casos de uso complexos • Pode detalhar a lógica de uma operação de uma classe • Swim lanes podem representar atores do caso de uso
Exemplo de aplicação	"Reservar Veículo" como funcionalidade do sistema, com ator Cliente	Classes: Cliente, Reserva, Veículo, Categoria — com atributos e operações	Fluxo: cliente informa dados → sistema verifica disponibilidade → [se disponível] registra reserva / [se não] sugere alternativas
Público-alvo	Clientes, usuários, gerentes, testadores (todos os envolvidos)	Desenvolvedores, projetistas, analistas (equipe técnica)	Desenvolvedores, analistas de negócio (quando usado para BPM), testadores